# DATA STRUCTURES. TERMINOLOGIES & BASICS

**Data:** Data is defined as simply a Value or set of Values of different types which is called data types like string, integer, char etc.,

**Structure:** It is a way of organising information for easy access and storage

**Program:** A set of Instructions

Data structures + Algorithms

Data structure = A container stores Data

Algorithm = Logic + Control

## Data Structure:

A Data structure is a systematic way of organising data in a Computer so that it can be used efficiently.

A Data structure is a scheme for organising related pieces of information to perform legal operations on the data.

## Need for Data Structure:

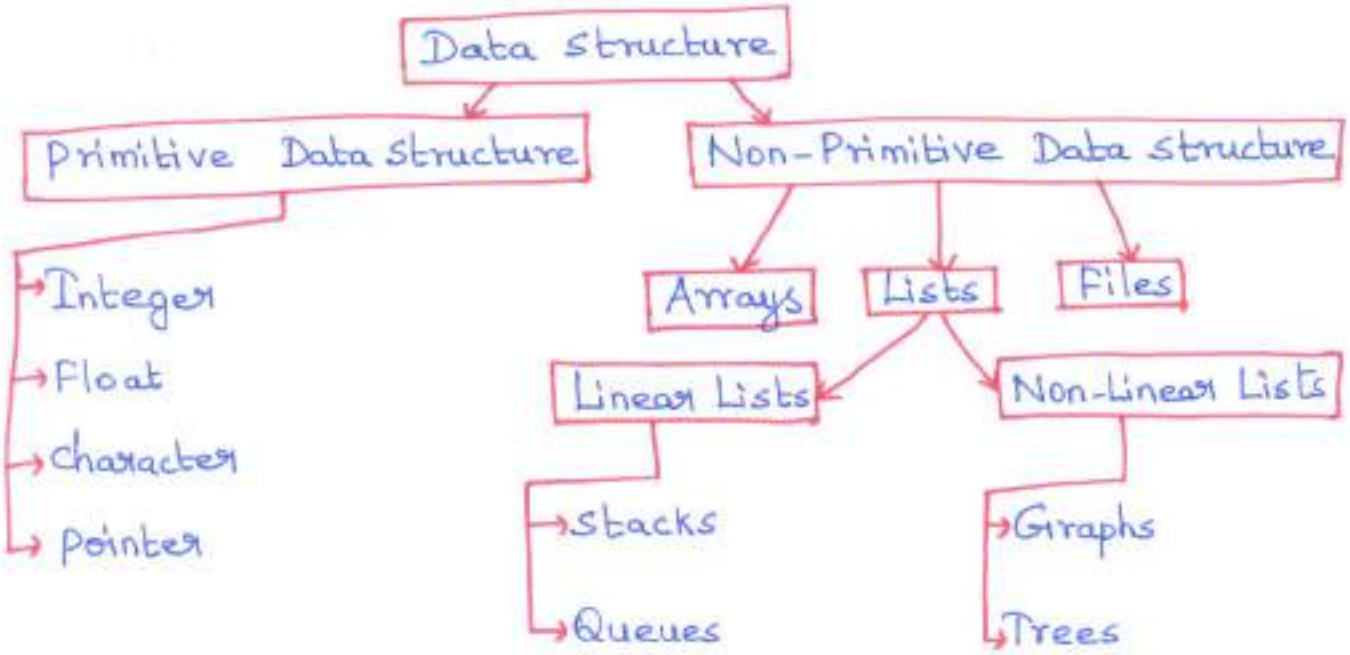
1. Data structure is the study of how data are stored in a computer so that operations can be implemented efficiently.
2. Data structures are especially important when large amount of information is involved.

3. Data structure is a conceptual and concrete ways to organize data for efficient storage and manipulation.

4. Data structure helps to understand the relationship of one data element with the other and organisation within the memory.

5. The data organisation is simple and clearly visualized.

## CLASSIFICATION OF DATA STRUCTURE




### Primitive Data structure:

A primitive data structure used to represent the standard data types of any one of the computer languages (integer, char, Float etc)

## Non- Primitive Data structure:

A Non-primitive data structure can be constructed with the help of any one of the primitive data structure.

* It can be a structure having specific functionality
* It can be designed by user
* It can be classified as Linear and Non-Linear Data structure.

## Linear Data structure:

A Linear data structure traverses the data elements sequentially, in which one data element can directly be reached.

Examples: Arrays, Linked Lists, stacks, Queues

## Non- Linear Data structure:

Every data item is attached to several other data items in a way (i.e) specific for reflecting relationships. The data items are not arranged in a sequential structure. It also follows hierarchial relationship.

Examples: Trees, Graphs, Heaps

## Operations on Data Structures:

* Traversal
* Insertion
* Deletion
* Searching
* Sorting
* Merging

The basic operations that are performed on data structures are as follows:

* Traversal - Traversal of a data structure means processing all the data elements present in it exactly once.

* Insertion - Insertion means addition of a new data element in a data structure.

* Deletion - Deletion means removal of a data element from a data structure if it is found.

* Searching - Searching involves searching for the specified data element in a data structure.

* Sorting - Arranging data elements of a data structure in a specified order is called sorting.

* Merging - Combining elements of two similar data structures to form a new data structure of the same type is called merging.

## Areas where data structures are applied extensively

1. Operating Systems
2. Compiler Design
3. Database Management System
4. Statistical Analysis Package
5. Numerical Analysis
6. Graphics
7. Artificial Intelligence
8. Simulation
9. Satellite Communication

# UNIT - I

## LINEAR DATA STRUCTURES - LIST

### ABSTRACT DATA TYPES (ADTs)

An abstract data type is a set of operations with associated values that are specified accurately, independent of any particular implementation.

* Abstract data types are mathematical abstractions which describes a container to hold a finite number of objects where objects may be associated through a given binary relationship.
* ADTs can be viewed as an extension of modular design.
* Objects such as list, sets and graphs along with their operations can be viewed as ADTs.
* Integers, reals and booleans have operations associated with them and so called as ADT's.
* For set ADT, operations such as union, intersection, Find, size and complements etc are referred as ADT's.
* The basic idea of Abstract Data Type is that the implementation of these operations is written once in the program, and any other part of the program that needs to perform an operation on the ADT can do so by calling the appropriate function.

## Modularity:

The basic rule in programming is that no routine should ever exceed a page. This is accomplished by breaking the program down into modules. Each module is a logical unit and does a specific job. Its size is kept small by calling other modules.

## Advantages:

1. It is much easier to debug small routines than large routines.
2. It is easier for several people to work on a modular program simultaneously.
3. A well-written modular program places certain dependencies in only one routine, making changes easier.

## The LIST ADT

A List or sequence is an abstract datatype that represents a countable number of ordered values, where the same value may occur more than once.

* A general list is of the form

$$A_1, A_2, A_3 \ldots\ldots A_N$$

$N \rightarrow$ length of the list
$A_1 \rightarrow$ First element
$A_N \rightarrow$ Last element
$A_i \rightarrow$ position $i$

* Rule 1 : $A_{i+1}$ follows / succeeds $A_i$ when $(i < N)$
* Rule 2 : $A_{i-1}$ precedes $A_i$ when $(i > 1)$

* The empty list is of size 0.

## Operations in the List ADT:

* PrintList() – prints the elements of the list
* MakeEmpty() – make the list empty
* Find() – returns the position of the first occurrence of a key
* Insert() – insert some key from some position in the list
* Delete() – delete some key from some position in the list
* FindKth() – returns the element in some position
* Next() – returns the position of successor
* Previous() – returns the position of predecessor.

## Implementation of LIST ADT:

The LIST ADT can be Implemented in three ways –

(i) Array Implementation

– An array is a random access data structure where each element can be accessed directly and in constant time.

(Eg) accessing a page in a book is independent of other pages.

(ii) Linked List Implementation

– A Linked List is a sequential access data structure, where each element can be accessed only in particular order.

(Eg) Roll of paper or tape.

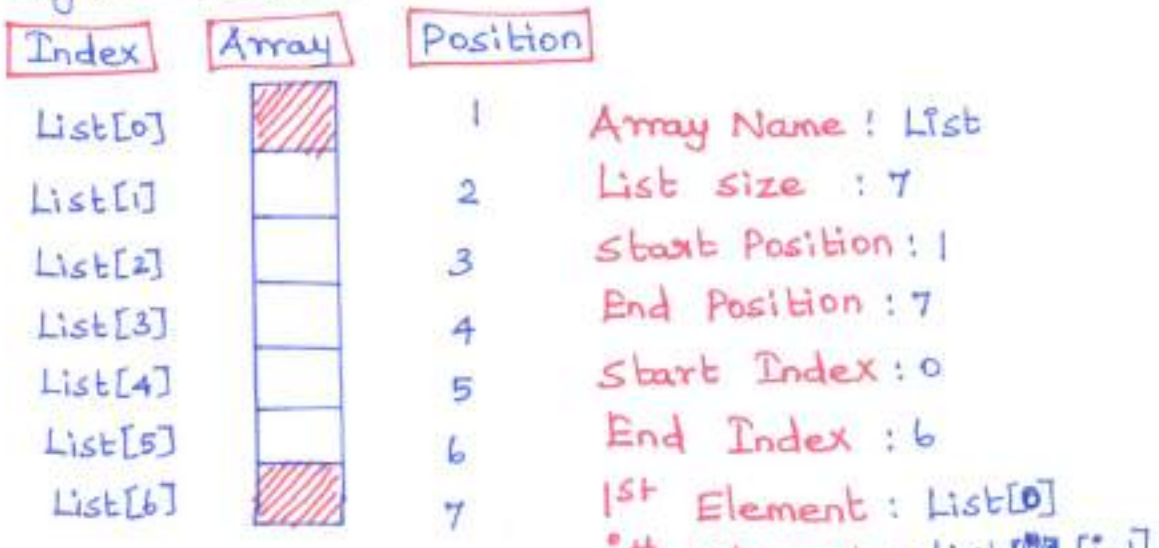
(iii) Cursor Implementation

- If linked lists are required and pointers are not available, then an alternate method is used called as cursor implementation.

(Eg) BASIC & FORTRAN languages do not support pointers.

## ARRAY IMPLEMENTATION OF LISTS ADT

* An array is a data structure which can store a fixed-size sequential collection of elements of the same type.
* A specific element in an array is accessed by an index.
* An array consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.




LIST STRUCTURE

## OPERATIONS

### 1. IsEmpty (LIST)

```
if (current size = = 0)
    print " List is empty"
else
    print " List is not empty"
```

### 2. Is Full (LIST)

```
if (current size = = Max size)
    print " List is Full"
else
    print " List is not Full"
```

### 3. Insert Element to End of the LIST

1. check whether List is full or not
   (i) If List is full
       return " Error Message"
   (ii) If List is not full
       a.) Get the position to insert the new element
           Position = current size + 1
       b.) Insert element to position
       c.) Insert the Current size by 1
           Current Size = Current Size + 1

### 4. Delete Element from End of the LIST

1. check whether List is empty or not
   (i) If List is empty
       return " Error Message"

(ii) If List is not empty

a.) Get the position of the element to delete

Position = current size

b.) Delete the element from the position

c.) Decrease the current size by 1

current size = current size - 1

5. Insert element to front of the List

1. Check whether the List is full or not

(i) If List is full

return " Error Message"

(ii) If List is not full

a.) Free the 1st position of the list by moving all the element to one position forward.

New position = current Position + 1

b.) Insert the element to the 1st position

c.) Increase the current size by 1

Current Size = Current Size + 1

6. Delete Element from front of the List

1. Check whether the list is empty or not

(i) If list is empty

return " Error Message"

(ii) If list is not empty

a.) move all the elements except one in the 1st position to one position backward.

New position = Current Position - 1

b.) After the 1st step, element in 1st position will be automatically deleted.

c.) Decrease the current size by 1.

Current size = Current size - 1.

## 7. Insert element to $n^{th}$ Position of the List

1. check whether the List is full or not

(i) If the List is full

return "Error Message"

(ii) If the List is not full

a.) If the List is empty, then

Insert element at position 1.

b.) If ($n^{th}$ position > current size)

return "$n^{th}$ position not available in the list"

c.) else

(i) free the $n^{th}$ position of the list by moving all elements to one position forward

New position = current Position + 1

(ii) Insert the element to $n^{th}$ position

(iii) Increase the current size by 1

Current size = Current size + 1

## 8. Delete Element from $n^{th}$ position of the List

1. check whether the List is Empty or not

(i) If the List is Empty

return "Error Message"

(ii) If List is not empty

a.) If ($n^{th}$ position > current size)

return " $n^{th}$ position not available in List"

b.) If ( $n^{th}$ position = = current size)

(i) Delete the element from $n^{th}$ position

(ii) Decrease the current size by 1.

Current size = Current size − 1

c.) If ($n^{th}$ position < current size)

(i) move all the elements to one position backward

New position = Current Position − 1

(ii) After the previous step, $n^{th}$ element will be deleted automatically

(iii) Decrease the current size by 1.

Current size = Current size − 1.

## 9. Search Element in the List

1. check whether the list is empty or not

(i) If List is empty

return " Error Message"

(ii) If List is not empty

a.) Find the position of last element

Last position = Current size

b.) for position 1 to Last position perform the following

(i) if( element @ position = = Search Element)

return " Search element in position"

(ii) else

return " No such element"

## 10. Print the Elements in the List

1. Check whether the list is empty or not

(i) If List is empty,

return "Error Message"

(ii) If List is not empty,

a.) Find the position of last element

Last position = Current size

b.) for position (1 to Lastposition)

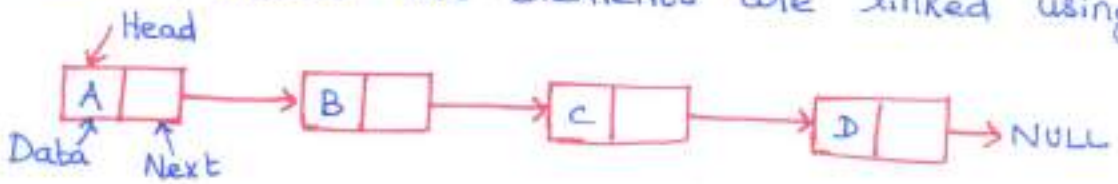
c.) print the position and element available in the list.

## Limitations of Arrays:

1. The size of the array is fixed. It should be known in advance to allocate memory even in dynamic allocation.
2. Inserting a new element in array is expensive, because existing elements should be shifted to create new memory location for the inserting element. So, the worst case of these operations is O(N).

## LINKED LIST IMPLEMENTATION OF LISTS:

* Like arrays, Linked List is a linear data structure
* In linked lists, elements are not stored at contiguous locations, instead the elements are linked using pointers.



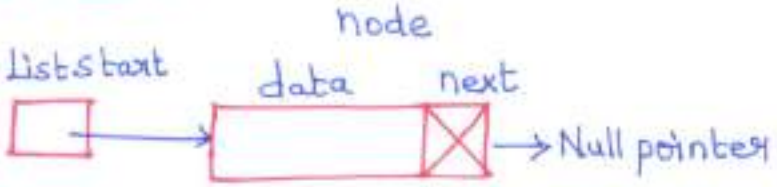

### Advantages over Arrays

1. Dynamic size
2. Ease of insertion/deletion

### Drawbacks

1. Random Access is not allowed. Accessing the element from first node is executed sequentially. So binary search cannot be implemented using linked lists.
2. Extra memory space for a pointer is required with each element of the list.

### Representation




* A linked list is represented by a pointer to the first node of the linked list.
* The first node is called head
* If the linked list is empty, then value of head is NULL.
* Each node contains subparts – data and pointer to next node

# ROUTINES OF LINKED LIST IMPLEMENTATION USING LISTS

```
# ifndef _List_H

Struct Node;
typedef struct Node *PtrToNode;
typedef PtrToNode List;
typedef PtrToNode Position;

List MakeEmpty (List L);
int IsEmpty (List L);
int IsLast (Position P, List L);
Position Find (ElementType X, List L);
Void Delete (ElementType X, List L);
Position FindPrevious (ElementType X, List L);
Void Insert (ElementType X, List L, Position P);
Void DeleteList (List L);
Position Header (List L);
Position First (List L);
Position Advance (Position P);
ElementType Retrieve (Position P);

# endif

Struct Node
{
   Element Type Element;
   Position Next;
};
```

```
int IsEmpty (List L)
{
  return  L->Next == NULL;
}

int IsLast (Position P, List L)
{
  return P->Next == NULL;
}

Position Find (ElementType X, List L)
{
  Position P;
  P = L->Next;
  while ( P != NULL && P->Element != X)
    P = P->Next;
    return P;
}

Position FindPrevious ( ElementType X, List L)
{
  Position P;
  P = L;
  while (P->Next != NULL && P->Next->Element != X)
   P = P->Next;
   return P;
}
```

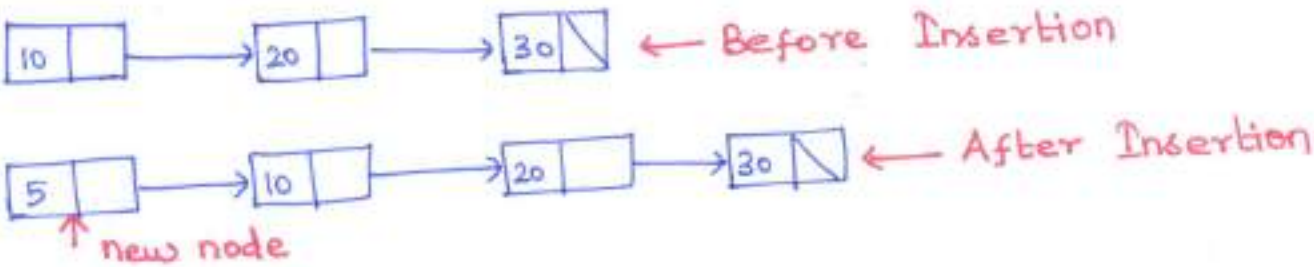
```
Void Insert ( ElementType X, List L, Position P)
{
   Position TmpCell;
   TmpCell = malloc ( sizeof ( struct Node));
   if ( TmpCell = = NULL)
   FatalError (" out of space! ");
   TmpCell → Element = X ;
   TmpCell → Next = P → Next;
   P → Next = TmpCell;
}

Void Delete ( ElementType X, List L)
{
   Position P, TmpCell;
   P = FindPrevious ( X, L);
   if ( ! IsLast ( P, L))
   {
     TmpCell = P → Next;
     P → Next = TmpCell → Next;
     free ( TmpCell);
   }
}

Void DeleteList ( List L)
{
   Position P, Tmp;
   P = L → Next;
   L → Next = NULL;
   while ( P! = NULL)
   {
     Tmp = P → Next;
     free (P);
     P = Tmp;
   }
}
```

# SINGLY LINKED LIST:

## INSERTION

### (i) Insertion at beginning of the list

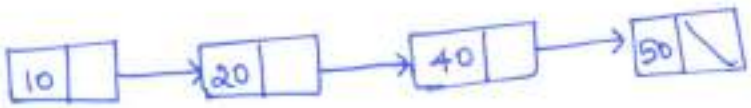
```
void insert_beg (struct node **q, int no)
{
  struct node * temp;
  temp = malloc (sizeof (struct node));
  temp -> data = no;
  temp -> next = *q;
  *q = temp;
}
```



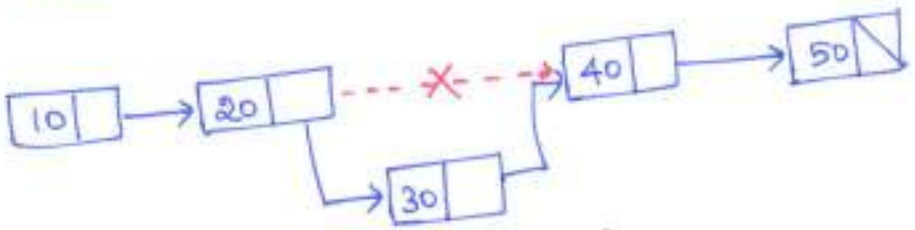

### (ii) Insertion after any specific node

```
void insert_after (struct node *q, int loc, int no)
{
  struct node *temp, *r;
  int t;
  temp = q;
  for (i=0; i< loc; i++)
  {
    temp = temp -> next;
    if ( temp = = NULL)
    {
      printf (" There are less than %d elements in list', doc)
    return;
  }
```

```
r = malloc (sizeof (struct (node));
r→data = no;
r→ next = temp → next;
temp → next = r;
}
```




← Before Insertion




← After Insertion

r = new node

(iii) Insertion at the end of the list

```
Void create ( struct node **q, int no)
{
  Struct node *temp, *r;
  if ( *q = = NULL)
  {
    temp = malloc( sizeof (struct node))
     temp →data = no;
     *q = temp;
  }
  else
  {
     temp = *q;
     while ( temp → next ! = NULL)
      temp = temp → next;
```

```
    r = malloc (sizeof (struct node));
    r->data  = no;
    r->Next = NULL;
    temp->next = r;
  }
}
```

## DELETION

```
void del(struct node **q, int no)
{
  Struct node *old, *temp;
   temp = *q;
   while( temp! = NULL)
   {
     if (temp->data = = no)
      {
        if (temp = = *q)
        *q = temp->next;
        else
        old->next = temp->next;
        free (temp);
        return;
      }
     else
     {
       old = temp;
       temp = temp->next;
      }
   }
    printf (" Element %d not found ", no);
}
```

## Displaying the contents of Linked List

```
Void display (struct node * start)
{
  while (start != NULL)
  {
    printf (" %d", start -> data);
    start = start -> next;
  }
}
```

## Counting the number of nodes in a linked list

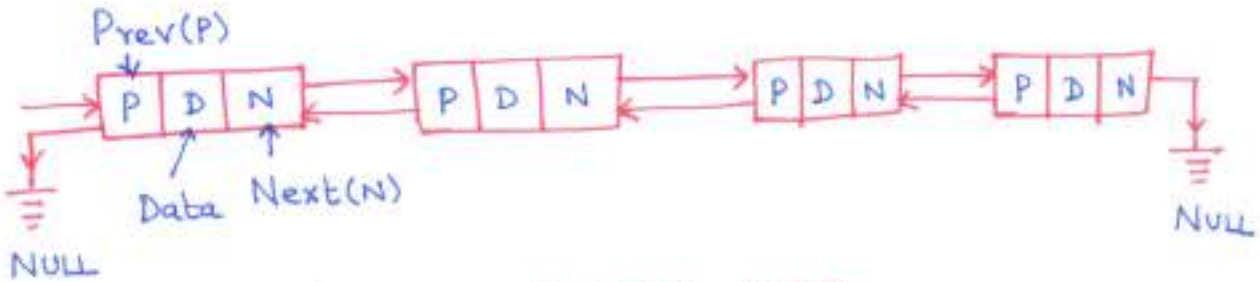
```
int count (struct node *q)
{
  int c=0;
  while(q != NULL)
  {
    q = q -> next;
    c++;
  }
 return c;
}
```

## Reversing the linked list

```
Void reverse (struct node **x)
{
  Struct node *q, *r, *s;
  q = *x;
  r = NULL;
  while (q != NULL)
  {
      s = r;
      r = q;
      q = q -> next;
      r -> next = s;
  }
  *x = r;
}
```

## DOUBLY LINKED LIST:

* The doubly or two way linked list uses double set of pointers – one pointing to the next item and other pointing to the preceding item.
* It can be traversed in two directions either from the beginning of the list to the end or from end of the list to the beginning.




DOUBLY LINKED LIST

* Each node contains three parts:
  1. An information field which contains the data
  2. A pointer field next which contains the location of the next node in the list
  3. A pointer field prev which contains the location of the preceding node in the list.
* The structure for DLL is

```
Struct Node
{
  int data;
  struct Node * prev;
  struct Node * next;
}
```

# INSERTION AT DOUBLY LINKED LIST

## (i) Insertion at the end of DLL

```
Void create(struct node **s, int num)
{
  Struct node *r,*q = *s;
  if(*s == NULL)
  {
    *s = malloc(sizeof(struct node));
    (*s) -> prev = NULL;
    (*s) -> data = num;
    (*s) -> next = NULL;
  }
  else
  {
    while(q -> next! = NULL)
     q = q -> next;
     r = malloc(sizeof(struct node));
     r -> data = num;
     r -> next = NULL;
     r -> prev = q;
     q -> next = r;
  }
}
```

## (ii) Insertion at beginning of DLL

```
Void begin(struct node **s, int num)
{
  struct node *q;
  q = malloc(sizeof(struct node));
  q->prev = NULL;
  q->data = num;
  q->next = *s;
  (*s)->prev = q;
}
```

## (iii) Insertion after specified node in DLL

```
Void after(struct node *q, int loc, int num)
{
 struct node *temp;
 int i;
 for(i=0; i< loc; i++)
 {
   q = q->next;
   if(q==NULL)
   {
     printf("There are less than %d Elements", loc);
     return;
   }
}
q = q->prev;
temp = malloc(sizeof(struct node));
temp->data = num;
temp->prev = q;
temp->next = q->next;
temp->next->prev = temp
q->next = temp;
}
```

## DELETION AT DLL:

```
Void delete ( struct node **s, int num)
{
  struct node *q = *s;
  while (q != NULL)
  {
    if(q->data == num)
    {
      if( q == *s)
      {
        *s = (*s) -> next;
       (*s) -> prev = NULL;
      }
     else
     {
       if(q -> next == NULL)
       q -> prev -> next = NULL;
     }
    else
    {
      q -> prev -> next = q -> next;
      q -> next -> prev = q -> prev;
    }
    free(q);
  }
  return;
 }
 q = q -> next;
}
printf(" %d not found", num);
}
```

## Advantages – DLL

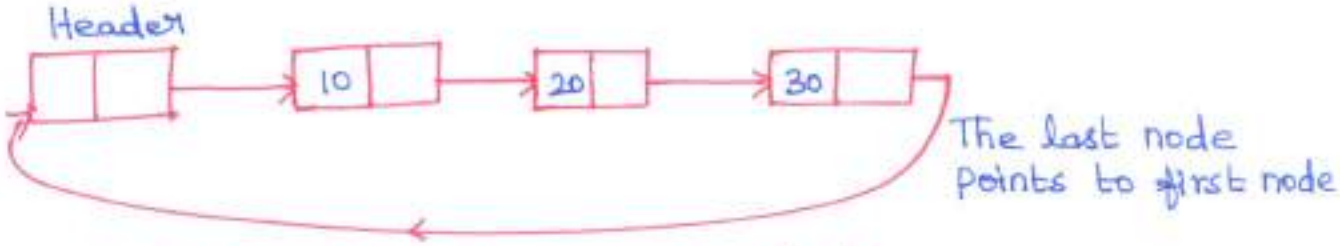
* Deletion operation is easier
* Finding the predecessor and successor of a node is easier.

## Disadvantage – DLL

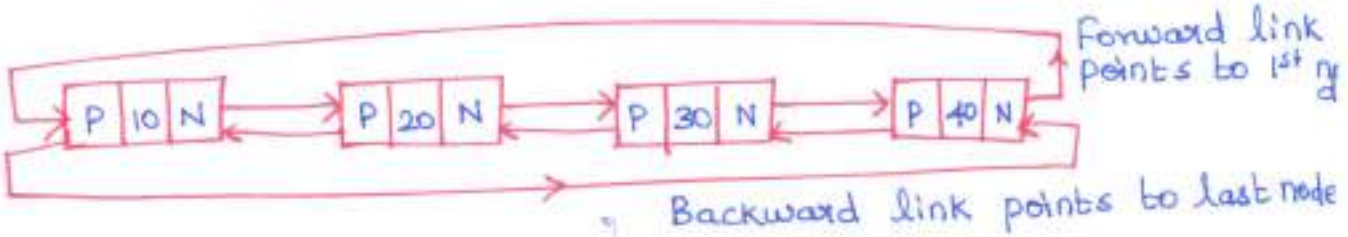
* Occupies more memory space since it has two pointers.

## CIRCULAR LINKED LIST

* In circular linked list, the pointer of the last node points to the first node.
* Circular Linked List can be Implemented as
  (i) Singly linked circular list
  (ii) Doubly linked circular list




Singly Linked Circular List




Doubly Linked Circular List

* Circular Linked List are useful for implementation of queue. It is also applied where the applications repeatedly go around the list such as the Operating System running multiple applications.

## Advantages – Circular Linked List

* It allows to traverse the list starting at any point.
* It allows quick access to the first and last node
* It allows to traverse the list in either direction.

## ARRAYS AND LINKED LIST COMPARISION

| ARRAY | LINKED LIST |
|---|---|
| (i) Size of the array is fixed | (i) Size of the list is dynamic |
| (ii) Insertion and deletion are difficult due to linearity | (ii) Insertion and deletion are easy due to linking the order |
| (iii) Occupies less memory than linked list | (iii) Occupies more memory due to pointer variables. |
| (iv) (eg) int a[10]; | (iv) eg: temp = malloc(sizeof(struct node)); |
| (v) Searching is easy | (v) Searching is difficult |

## Dummy Header:

* It is the first node in the linked list before the actual data nodes.
* Header node value is referred as '0' and called as sentinel or it stores the number of nodes in the linked list.

## Advantages of Linked List:

1. The number of nodes in the linked list need not be specified in advance and so dynamic in nature.
2. It can grow or shrink in size depending upon the insertion or deletion operation.
3. Insertions and deletions at any place can be handled efficiently
4. It does not waste any memory space

## Disadvantages of Linked List:

1. Searching a node in Linked list is difficult and time consuming.
2. Requires additional memory space for storing the pointers.

## Applications of Linked List / List:

1. Polynomial ADT
2. Radix sort
3. Multi List

# POLYNOMIAL MANIPULATION

* A polynomial P(x) is an expression contains more than two terms of the form

$$ax^n + bx^{n-1} + \cdots + jx + k$$

where a, b, j, k are real numbers

n → degree of polynomial / non-negative number.

* A polynomial may be represented using arrays or linked list. The structure of polynomial term comprises of two parts – coefficient and exponent.

(Eg) $P(x) = 4x^3 + 6x^2 + 7x + 9$

## Polynomial Structure

```
struct poly
{
  int coeff;
  int exp;
  struct poly *next;
} *list1, *list2, *list3;
```

## Polynomial creation

```
polycreate (poly *head1, poly *newnode1)
{
  poly *ptr;
  if (head1 == NULL)
  {
    head1 = newnode1;
    return (head1);
  }
```

```
    else
    {
      ptr = head1;
      while( ptr -> next! = NULL)
      ptr = ptr -> next;
      ptr -> next = newnode1;
    }
    return (head1);
}
```

## Addition of two polynomial

```
void add()
{
  poly *ptr1, *ptr2, *newnode;
  ptr1 = list1;
  ptr2 = list2;
  while (ptr1 ! = NULL && ptr2! =NULL)
  {
    newnode = malloc( sizeof (struct poly));
    if ( ptr1 -> exp = = ptr2 -> exp)
    {
      newnode -> coeff = ptr1 -> coeff + ptr2 -> coeff;
      newnode -> exp = ptr1 -> exp;
      newnode -> next = NULL;
      list3 = create ( list3, newnode);
      ptr1 = ptr1 -> next;
      ptr2 = ptr2 -> next;
    }
```

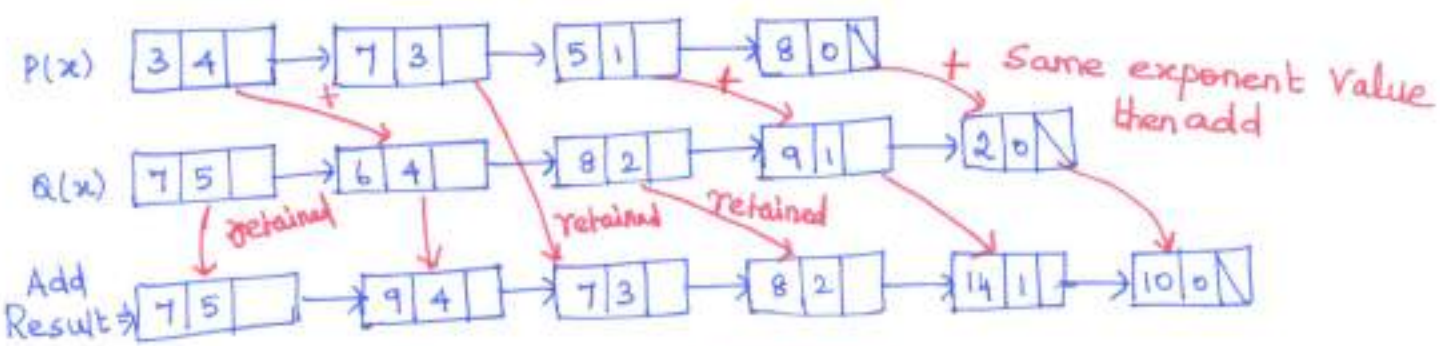
```
else
{
  if ( ptr1 → exp > ptr2 → exp)
  {
    newnode → coeff = ptr1 → coeff;
    newnode → exp   = ptr1 → exp;
    newnode → next  = NULL;
    list3 = create ( list3, newnode);
    ptr1 = ptr1 → next;
  }
 else
  {
    newnode → coeff = ptr2 → coeff;
    newnode → exp   = ptr2 → exp;
    newnode → next  = NULL;
    list3 = create ( list3, newnode);
    ptr2 = ptr2 → next;
  }
 }
}
```

Example:

$P(x) = 3x^4 + 7x^3 + 5x^1 + 8x^0$

$Q(x) = 7x^5 + 6x^4 + 8x^2 + 9x^1 + 2x^0$

$P(x) + Q(x) = 7x^5 + 9x^4 + 7x^3 + 8x^2 + 14x^1 + 10x^0$




POLYNOMIAL ADDITION